EE-170L Computer Programming Lab

Laboratory Report — No. 02

Basic Syntax, Input/Output, Data Types, Indentation, Comments & Arithmetic Operators in C++

Student Name	Zawar Shah
Registration No.	BF25NWELE0677
Course	EE-170L – Computer Programming Lab
Lab No.	02
Topic	Basic Syntax, I/O, Data Types, Indentation, Comments & Arithmetic Operators
Semester	Spring 2026
Institution	UET Peshawar

1. Lab Objectives

The goal of this lab is to familiarize students with the foundational elements of C++. By the end of this session, students will be able to:

- Set up a basic C++ program structure using the standard boilerplate.
- Manage user input and output using cin and cout.
- Apply correct data types for different kinds of variables.
- Follow indentation standards for readable and maintainable code.
- Write meaningful comments using single-line (//) and multi-line (/* */) syntax.
- Perform calculations using all standard arithmetic operators.

2. Theoretical Background

2.1 Basic Syntax, Input, and Output

Every C++ program follows a specific boilerplate structure. The iostream library facilitates communication between the program and the user. Three key elements are used:

- **Used to print data to the screen via the << operator.**cout (Character Output):
- **Used to read data from the keyboard via the >> operator.**cin (Character Input):
- **Stream insertion and extraction operators respectively.**<< and >>:

2.2 Data Types

C++ is a statically-typed language, meaning the type of every variable must be declared before use. Common data types are shown in the table below:

Data Type	Description	Example
int	Integers (whole numbers)	`int age = 20;`
double	Floating-point numbers (decimals)	`double price = 19.99;`
char	Single characters	`char grade = 'A';`
bool	Boolean values	`bool isPassed = true;`
string	Sequence of characters	`string text = "C++";`

2.3 Arithmetic Operators

C++ supports the following standard arithmetic operators:

- + (Addition) — Adds two operands.
- - (Subtraction) — Subtracts the right operand from the left.
- * (Multiplication) — Multiplies two operands.
- / (Division) — Divides; use double for decimal precision.
- % (Modulus) — Returns the remainder of integer division.

2.4 Indentation and Comments

Good coding practice requires consistent indentation and documentation:

- **Always indent code inside curly braces {} using 4 spaces or 1 tab to keep logic readable.**Indentation:
- **Use // for brief inline or line-level notes.**Single-line comments:
- **Use /* ... */ for longer block explanations.**Multi-line comments:

3. Example Programs

3.1 Hello World & User Greeting

The following program demonstrates basic output, variable declaration, and user input:

```
#include <iostream>
using namespace std;

int main() {
    // Basic Output
    cout << "Welcome to the C++ Lab!" << endl;

    string name;
    cout << "Enter your name: ";

    // Basic Input
    cin >> name;
```

```
    cout << "Hello, " << name << "! Let's start coding." << endl;

    return 0;
}
```

3.2 Simple Calculator

This program takes two integers as input and performs all five arithmetic operations:

```
#include <iostream>
using namespace std;

int main() {
    // Variable Declaration
    int num1, num2;

    cout << "Enter two integers: ";
    cin >> num1 >> num2;

    // Performing Arithmetic Operations
    int sum     = num1 + num2;
    int diff    = num1 - num2;
    int product = num1 * num2;

    /* Using double for division to ensure
       decimal precision if needed */
    double quotient  = (double)num1 / num2;
    int    remainder = num1 % num2;

    // Displaying Results
    cout << "Sum: "       << sum       << endl;
    cout << "Difference: "<< diff      << endl;
    cout << "Product: "   << product   << endl;
    cout << "Quotient: "  << quotient  << endl;
    cout << "Remainder: " << remainder << endl;

    return 0;
}
```

4. Lab Tasks

The following tasks were assigned to reinforce the concepts covered in this lab:

Task 1 — Variable Memory Size, Location, and Values

Write a C++ program that declares variables of different types and displays their memory size (using sizeof()), memory address (using &), and current values.

```
#include <iostream>
using namespace std;

int main() {
```

```
    int    a   = 10;
    double b   = 3.14;
    char   c   = 'Z';
    bool   d   = true;

    cout << "int    -> Value: " << a << "  Size: " << sizeof(a) << " bytes
Address: " << &a << endl;
    cout << "double -> Value: " << b << "  Size: " << sizeof(b) << " bytes
Address: " << &b << endl;
    cout << "char   -> Value: " << c << "  Size: " << sizeof(c) << " byte
Address: " << (void*)&c << endl;
    cout << "bool   -> Value: " << d << "  Size: " << sizeof(d) << " byte
Address: " << &d << endl;

    return 0;
}
```

Task 2 — Floating-Point Variable 'pi'

Declare and display a floating-point variable called pi with the value 3.14:

```
#include <iostream>
using namespace std;

int main() {
    double pi = 3.14;
    cout << "Value of pi = " << pi << endl;
    return 0;
}
```

Task 3 — Celsius to Fahrenheit Converter

Write a C++ program that reads a temperature in Celsius and converts it to Fahrenheit using the formula: F = (C × 9/5) + 32.

```
#include <iostream>
using namespace std;

int main() {
    double celsius, fahrenheit;

    cout << "Enter temperature in Celsius: ";
    cin  >> celsius;

    fahrenheit = (celsius * 9.0 / 5.0) + 32;

    cout << celsius << " C = " << fahrenheit << " F" << endl;

    return 0;
}
```

Task 4 — Additional Task Sheet

A separate task sheet will be provided by the instructor for further exercises covering the concepts introduced in this lab.

5. Conclusion

This lab provided a hands-on introduction to the foundational building blocks of C++ programming. Through the examples and tasks, the following key concepts were practiced:

- The standard boilerplate structure of a C++ program.
- Using cout and cin for output and input respectively.
- Declaring and using variables with appropriate data types (int, double, char, bool, string).
- Applying all five arithmetic operators to perform calculations.
- Writing clean, well-indented code with meaningful comments.
- Determining variable memory size and address using sizeof() and the address-of operator (&).

Mastering these fundamentals is essential for building more complex C++ programs in subsequent labs.